

R4DS 09 – Strings & Regular Expressions

Creating Strings

String Basics

Create strings with " (preferred) or ':

```
string1 <- "This is a string"  
string2 <- 'To include a "quote", use single quotes'
```

Escape sequences with \:

```
double_quote <- "\""  
backslash <- "\\ "  
x <- c("one\\ntwo", "one\\ttwo", "\\u00b5")  
str_view(x)
```

```
#> [1] | one  
#>      | two  
#> [2] | one{\\t}two  
#> [3] | μ
```

Raw Strings and Special Characters

Raw strings avoid escape headaches — start with `r"(` and end with `)"`:

```
tricky <- r"(She said "It's \"weird\\")"  
str_view(tricky)
```

```
#> [1] | She said "It's \"weird\\"
```

Common special characters:

Escape	Meaning
<code>\n</code>	Newline
<code>\t</code>	Tab
<code>\\</code>	Literal backslash
<code>\"</code>	Literal quote
<code>\uXXXX</code>	Unicode character

Exercises (Creating Strings)

- 1 Create strings containing:
 - He said "That's amazing!"
 - `\a\b\c\d`
 - `\\\\\\`
- 2 What does `str_view()` show for `"This\u00a0is\u00a0tricky"`? What is `\u00a0`?

Solutions (Creating Strings)

1

```
str_view('He said "That\'s amazing!"')
```

```
#> [1] | He said "That's amazing!"
```

```
str_view("\\a\\b\\c\\d")
```

```
#> [1] | \a\b\c\d
```

```
str_view("\\\\\\\\\\\\\\\\\\\\")
```

```
#> [1] | \\\\\\\
```

- 2 \u00a0 is a non-breaking space. Looks like a space but prevents line breaks. `str_view()` highlights it distinctly.

String Operations

str_c() — Combining Strings

```
str_c("x", "y", "z")
```

```
#> [1] "xyz"
```

```
str_c("Hello ", c("John", "Susan"))
```

```
#> [1] "Hello John" "Hello Susan"
```

Propagates NAs (unlike `paste0()`):

```
df <- tibble(name = c("Flora", "David", "Terra", NA))  
df |> mutate(greeting = str_c("Hi ", name, "!"))
```

```
#> # A tibble: 4 x 2
```

```
#>   name greeting
```

```
#>   <chr> <chr>
```

```
#> 1 Flora Hi Flora!
```

```
#> 2 David Hi David!
```

```
#> 3 Terra Hi Terra!
```

```
#> ... [output truncated]
```


str_glue() — Template Strings

Anything inside `{ }` is evaluated:

```
df |> mutate(greeting = str_glue("Hi {name}!"))
```

```
#> # A tibble: 4 x 2
#>   name greeting
#>   <chr> <glue>
#> 1 Flora Hi Flora!
#> 2 David Hi David!
#> 3 Terra Hi Terra!
#> ... [output truncated]
```

Escape literal braces by doubling: `{{ }` and `}}`.

`str_flatten()` collapses a vector into one string (pairs with `summarize()`):

```
str_flatten(c("x", "y", "z"), ", ")
```

```
#> [1] "x, y, z"
```

```
str_flatten(c("x", "y", "z"), ", ", last = ", and ")
```

```
#> [1] "x, y, and z"
```

Exercises (String Operations)

- 1 Compare `paste0()` with `str_c()` for: `str_c("hi ", NA)` and `str_c(letters[1:2], letters[1:3])`.
- 2 What's the difference between `paste()` and `paste0()`? How to recreate `paste()` with `str_c()`?
- 3 Convert: `str_c("The price of ", food, " is ", price)` to `str_glue()`.

Solutions (String Operations)

- 1 `str_c("hi ", NA)` returns `NA`; `paste0()` returns `"hi NA"`. `str_c(letters[1:2], letters[1:3])` gives an error (length mismatch); `paste0()` recycles silently.
- 2 `paste()` separates with a space by default; `paste0()` uses no separator. Recreate: `str_c(..., sep = " ")`.
- 3 `str_glue("The price of {food} is {price}")`

Extracting Data from Strings

Separating into Rows

`separate_longer_delim()` splits a string into rows by delimiter:

```
df1 <- tibble(x = c("a,b,c", "d,e", "f"))  
df1 |> separate_longer_delim(x, delim = ",")
```

```
#> # A tibble: 6 x 1  
#>   x  
#>   <chr>  
#> 1 a  
#> 2 b  
#> 3 c  
#> 4 d  
#> 5 e  
#> ... [output truncated]
```

`separate_longer_position()` splits by character width:

```
df2 <- tibble(x = c("1211", "131", "21"))  
df2 |> separate_longer_position(x, width = 1)
```

```
#> # A tibble: 9 x 1  
#>   x  
#>   <chr>  
#> 1 1
```

Separating into Columns

`separate_wider_delim()` creates new columns:

```
df3 <- tibble(x = c("a10.1.2022", "b10.2.2011",  
                    "e15.1.2015"))  
  
df3 |>  
  separate_wider_delim(x, delim = ".",  
                        names = c("code", "edition", "year"))
```

```
#> # A tibble: 3 x 3  
#>   code edition year  
#>   <chr> <chr>   <chr>  
#> 1 a10    1      2022  
#> 2 b10    2      2011  
#> ... [output truncated]
```

`separate_wider_position()` splits by fixed widths:

```
df4 <- tibble(x = c("202215TX", "202122LA",  
                    "202325CA"))  
  
df4 |> separate_wider_position(x,  
                               widths = c(year = 4, age = 2, state = 2))
```

```
#> # A tibble: 3 x 3  
#>   year age state
```

Handling Mismatches

When pieces don't match expected count, use `too_few` and `too_many`:

```
df <- tibble(a = c("1-1-1", "1-1-2", "1-3", "1"))
df |>
  separate_wider_delim(a, delim = "-",
    names = c("x", "y", "z"),
    too_few = "align_start")
```

```
#> # A tibble: 4 x 3
#>   x     y     z
#>   <chr> <chr> <chr>
#> 1 1     1     1
#> 2 1     1     2
#> 3 1     3    <NA>
#> 4 1    <NA> <NA>
#> ... [output truncated]
```

Options: `"debug"` (diagnose), `"align_start"` / `"align_end"` (fill NAs). For `too_many`: `"drop"` or `"merge"`.

Working with Letters

str_length() and str_sub()

```
str_length(c("a", "R for data science", NA))
```

```
#> [1]  1 18 NA
```

str_sub() extracts by position (negative = from end):

```
x <- c("Apple", "Banana", "Pear")
```

```
str_sub(x, 1, 3)
```

```
#> [1] "App" "Ban" "Pea"
```

```
str_sub(x, -3, -1)
```

```
#> [1] "ple" "ana" "ear"
```

Won't fail if string is too short:

```
str_sub("a", 1, 5)
```

```
#> [1] "a"
```

Case Conversion and Locale

```
str_to_upper("hello world")
```

```
#> [1] "HELLO WORLD"
```

```
str_to_lower("HELLO WORLD")
```

```
#> [1] "hello world"
```

```
str_to_title("hello world")
```

```
#> [1] "Hello World"
```

Encoding: UTF-8 is the modern standard. Use `locale` argument if reading non-UTF-8 data:

```
read_csv(file, locale = locale(encoding = "Latin1"))
```

Locale matters for case and sorting in non-English text.

Exercises (Letters)

- 1 Use `str_length()` and `str_sub()` to extract the middle letter of each name in `c("Alice", "Bob", "Catherine")`. What if even length?
- 2 What does `str_sub("abc", 1, 5)` return? Why?

Solutions (Letters)

❶

```
x <- c("Alice", "Bob", "Catherine")  
mid <- ceiling(str_length(x) / 2)  
str_sub(x, mid, mid)
```

```
#> [1] "i" "o" "e"
```

For even-length strings, this picks the left-of-center letter.

❷ Returns "abc" — `str_sub()` silently returns as much as possible without error.

Regular Expression Basics

Pattern Matching with `str_view()`

Literal characters match exactly:

```
str_view(fruit, "berry")
```

```
#> [6] | bil<berry>  
#> [7] | black<berry>  
#> [10] | blue<berry>  
#> [11] | boysen<berry>  
#> ... [output truncated]
```

The dot `.` matches any character:

```
str_view(c("a", "ab", "ae", "bd", "ea"), "a.")
```

```
#> [2] | <ab>  
#> [3] | <ae>
```

Quantifiers

Quantifier	Meaning
?	0 or 1 (optional)
+	1 or more
*	0 or more
{n}	Exactly n
{n,}	At least n
{n,m}	Between n and m

```
str_view(c("a", "ab", "abb"), "ab?")
```

```
#> [1] | <a>  
#> [2] | <ab>  
#> [3] | <ab>b
```

```
str_view(c("a", "ab", "abb"), "ab+")
```

```
#> [2] | <ab>  
#> [3] | <abb>
```

Character Classes and Alternation

`[]` defines a set; `[^]` negates it:

```
str_view(words, "[aeiou]x[aeiou]")
```

```
#> [284] | <exa>ct  
#> [285] | <exa>mple  
#> [288] | <exe>rcise  
#> ... [output truncated]
```

```
str_view(words, "[^aeiou]y[^aeiou]")
```

```
#> [836] | <sys>tem  
#> [901] | <typ>e
```

Alternation with `|`:

```
str_view(fruit, "apple|melon|nut")
```

```
#> [1] | <apple>  
#> [13] | canary <melon>  
#> [20] | coco<nut>  
#> [52] | <nut>  
#> ... [output truncated]
```


Key Regex Functions

str_detect() and str_count()

str_detect() returns logical — pairs with filter():

```
str_detect(c("apple", "banana", "pear"), "an")
```

```
#> [1] FALSE TRUE FALSE
```

```
x <- c("apple", "banana", "pear")  
str_count(x, "a")
```

```
#> [1] 1 3 1
```

str_count() with mutate():

```
df |> mutate(  
  vowels = str_count(name, "[aeiouAEIOU]")
```

str_replace() and str_remove()

```
x <- c("apple", "pear", "banana")  
str_replace_all(x, "[aeiou]", "-")
```

```
#> [1] "-ppl-" "p--r"  "b-n-n-"
```

```
str_remove_all(x, "[aeiou]")
```

```
#> [1] "ppl" "pr"  "bnn"
```

`str_replace()` replaces first match only; `str_replace_all()` replaces all.

`str_remove()` / `str_remove_all()` are shortcuts for replacing with "".

separate_wider_regex()

Extract data using regex patterns:

```
df <- tribble(
  ~str,
  "<Sheryl>-F_34", "<Kisha>-F_45",
  "<Brandon>-N_33", "<Justin>-M_41",
)
df |> separate_wider_regex(str, patterns = c(
  "<", name = "[A-Za-z]+", ">-",
  gender = ".", "_", age = "[0-9]+"))
```

```
#> # A tibble: 4 x 3
#>   name      gender age
#>   <chr>    <chr>  <chr>
#> 1 Sheryl  F         34
#> 2 Kisha   F         45
#> 3 Brandon N         33
#> ... [output truncated]
```

Named patterns become columns; unnamed patterns are consumed silently.

Exercises (Key Functions)

- 1 Replace all forward slashes in `"a/b/c/d/e"` with backslashes.
- 2 Implement a simple `str_to_lower()` using `str_replace_all()`.
- 3 Create a regex that matches US telephone numbers (e.g., 123-456-7890).

Solutions (Key Functions)

❶

```
str_replace_all("a/b/c/d/e", "/", "\\")
```

```
#> [1] "a\\b\\c\\d\\e"
```

❷

```
str_replace_all(x,  
  set_names(letters, LETTERS))
```

❸ "\\d{3}[-]?\\d{3}[-]?\\d{4}" or "\\((?\\d{3}\\))?[-]?\\d{3}[-]?\\d{4}" (with optional parens).

Pattern Details

Escaping Metacharacters

To match literal `.`, `?`, `+`, etc., escape with `\\`:

```
str_view(c("abc", "a.c", "bef"), "a\\.c")
```

```
#> [2] | <a.c>
```

To match literal `\`:

```
x <- "a\\b"  
str_view(x, "\\")
```

```
#> [1] | a<\>b
```

Alternative: use `fixed()` to disable regex entirely:

```
str_view(c("", "a", "."), fixed("."))
```

```
#> [3] | <.>
```


Anchors

`^` matches start, `$` matches end:

```
str_view(fruit, "^a")
```

```
#> [1] | <a>ppl  
#> [2] | <a>pric  
#> [3] | <a>voca  
#> ... [output truncated]
```

```
str_view(fruit, "a$")
```

```
#> [4] | banan<a>  
#> [15] | cherimoy<a>  
#> [30] | feijo<a>  
#> ... [output truncated]
```

Match full string: `^apple$`. Word boundary: `\\b`.

```
x <- c("summary(x)", "summarize(df)", "sum(x)")  
str_view(x, "\\bsum\\b")
```

```
#> [3] | <sum>(x)
```

Shorthand Character Classes

Shorthand	Matches	Inverse
<code>\d</code>	Digits	<code>\D</code>
<code>\s</code>	Whitespace	<code>\S</code>
<code>\w</code>	Word chars (letters, digits)	<code>\W</code>

```
x <- "abc 123 !@#"
str_view(x, "\\d+")
```

```
#> [1] | abc <123> !@#
```

```
str_view(x, "\\w+")
```

```
#> [1] | <abc> <123> !@#
```

Ranges: `[a-z]`, `[A-Z]`, `[0-9]`.

Grouping and Back References

Parentheses create **capturing groups**:

```
# Repeated pair of letters  
str_view(fruit, "(..)\\1")
```

```
#> [4] | b<anan>a  
#> [20] | <coco>nut  
#> [22] | <cucu>mber  
#> ... [output truncated]
```

Use in `str_replace()` to reorder:

```
sentences |>  
  str_replace("(\\w+) (\\w+) (\\w+)",  
              "\\1 \\3 \\2") |>  
  head(3)
```

```
#> [1] "The canoe birch slid on the smooth planks."  
#> [2] "Glue sheet the to the dark blue background."  
#> [3] "It's to easy tell the depth of a well."  
#> ... [output truncated]
```

Non-capturing group: `(?:)`.

`regex()` wraps pattern with options:

```
bananas <- c("banana", "Banana", "BANANA")  
str_view(bananas,  
          regex("banana", ignore_case = TRUE))
```

```
#> [1] | <banana>
```

```
#> [2] | <Banana>
```

```
#> [3] | <BANANA>
```

Other flags: `multiline` (`^/$` per line), `dotall` (`.` matches `\n`), `comments` (allow whitespace/comments in pattern).

Exercises (Pattern Details)

- 1 Create regexes to find words in `stringr::words` that:
 - Start with “y”
 - End with “x”
 - Are exactly three letters long
 - Have seven letters or more
- 2 Describe what these match: `^.*$, \d{4}-\d{2}-\d{2}, (..)\1`.
- 3 Switch first and last letters of `words`. Which are still valid words?

Solutions (Pattern Details) — 1

```
str_view(words, "^y")      # start with y
```

```
#> [975] | <y>ear  
#> [976] | <y>es  
#> [977] | <y>esterday  
#> ... [output truncated]
```

```
str_view(words, "x$")      # end with x
```

```
#> [108] | bo<x>  
#> [747] | se<x>  
#> [772] | si<x>  
#> ... [output truncated]
```

```
str_view(words, "^...$")    # exactly 3 letters
```

```
#> [9] | <act>  
#> [12] | <add>  
#> [22] | <age>  
#> ... [output truncated]
```

```
str_view(words, "^.{7,}$")  # 7+ letters
```

```
#> [4] | <absolute>
```

- ② `^.*$` matches any entire string. `\d{4}-\d{2}-\d{2}` matches dates like 2024-03-31. `(..)\1` matches repeated two-character sequences (e.g., “anan” in banana).

③

```
swapped <- str_replace(words, "^(..)(.*)"(.)",  
                        "\\3\\2\\1")  
intersect(swapped, words)
```

```
#> [1] "a"          "america"    "area"  
#> [4] "dad"        "dead"       "lead"  
#> [7] "read"       "depend"     "god"  
#> [10] "educate"    "else"       "encourage"  
#> [13] "engine"     "europe"     "evidence"  
#> ... [output truncated]
```